

Take-Home Test: E-commerce API System

Objective

Design and implement a **simple e-commerce backend system** focusing on **API design**, **database schema**, and **system architecture**.

The goal is not to deliver a production-ready system but to showcase your ability to structure a scalable backend, model data relationships, and expose clean, maintainable APIs.

Functional Requirements

1. User Management

- Users can **register** and **log in**.
- Use **JWT-based authentication**.
- Roles: customer, admin.
 - Admins can manage products.
 - Customers can browse and purchase products.

2. Products

- Admins can **create, update, delete** products.
- Fields: id, name, description, price, stockQuantity, categoryId, createdAt, updatedAt.
- Customers can **list** and **view** product details.
- Products should support **pagination & filtering** (by category, price range).

3. Shopping Cart

- Customers can add/remove products to/from their cart.
- A cart is tied to a user session.
- Cart should show: product list, quantity, subtotal, and total price.

4. Orders

- Customers can place an order from their cart.
 - Order should:
 - Deduct product stock.
 - Store order details (items, quantities, total price, timestamp).
 - Customers can view **order history**.
 - Admins can view **all orders**.
-



Non-Functional Requirements

- Implement using **Node.js** with a framework (Express.js, NestJS, or Fastify).
 - Use a database (SQL or NoSQL). Show a **clear schema design**.
 - Provide **API documentation** (OpenAPI/Swagger or simple markdown).
 - Implement **basic error handling** (e.g., invalid stock, unauthorized access).
 - Demonstrate **scalable architecture** (service layer, repository pattern, controllers, etc.).
-



Deliverables

1. **Source code** (GitHub repo or zipped folder).
 2. **Database schema design** (ERD diagram or schema.sql file).
 3. **README.md** including:
 - Setup instructions.
 - API endpoints with example requests/responses.
 - Short explanation of architecture decisions.
 - Assumptions & trade-offs.
 4. (Optional, bonus) Unit/integration tests for critical APIs.
-

Time Expectation

We don't expect candidate spend more than ~6 hours for this test

Evaluation Criteria

- **API Design:** RESTful, consistent, and easy to use.
- **Database Design:** proper normalization, relationships, indexing considerations.
- **System Architecture:** clean separation of concerns, scalability in mind.
- **Code Quality:** modularity, readability, best practices.
- **Error Handling:** how gracefully the system manages edge cases.
- **Documentation:** clear instructions and API usage.
- **Testing (bonus):** demonstrates ability to validate business logic.